

智慧農業人工智慧物聯網程式 撰寫 與實務應用

林聖泉

2026/8/5

課程大綱

- 準備工作
- AI協作程式設計
- Thonny軟體
- Python程式設計
- ESP32-S3
- MQTT
- 藍牙低功耗

準備工作

- 請按這裡加入文字

課程宗旨

- 以問題為導向，藉由人工智慧Microsoft Copilot協作產生程式範本，再依據實際需求修改程式
- 養成撰寫Python程式能力，了解如何修改程式
- 運用ESP32-S3開發板建立農用物聯網
- 盡量不涉及複雜語法，透過AI進一步探究相關內容
- 善用AI協作，降低物聯網開發門檻

軟體安裝

- Microsoft Copilot : AI協作程式設計
- Thonny : Python程式編輯、編譯、燒錄
- Mosquitto : MQTT伺服器
- 手機 nRF Connect APP : 藍牙通訊

器材清單-1

- ESP32-S3 SuperMini開發板



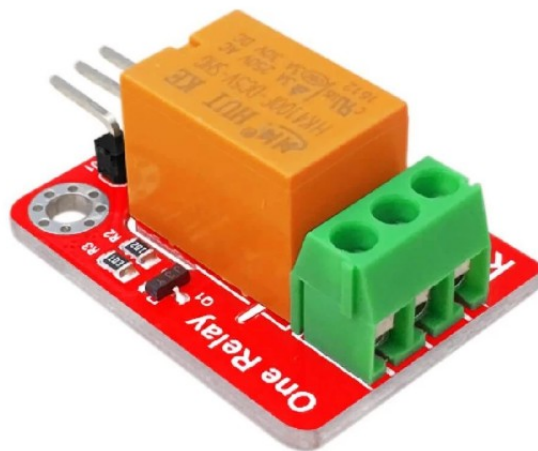
器材清單-2

- AM2120 AOSONG 奧松高精度溫濕度感測器



器材清單-3

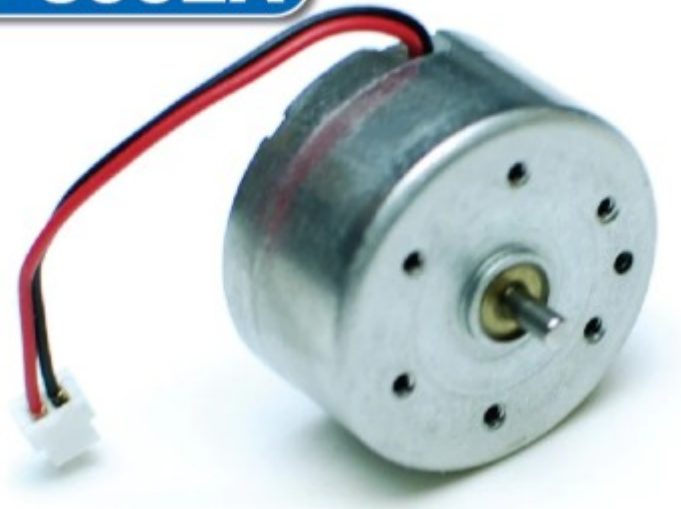
- PWM servor motor : MG995 舵機 伺服馬
達 180度
- Relay module



器材清單-4

- DC馬達

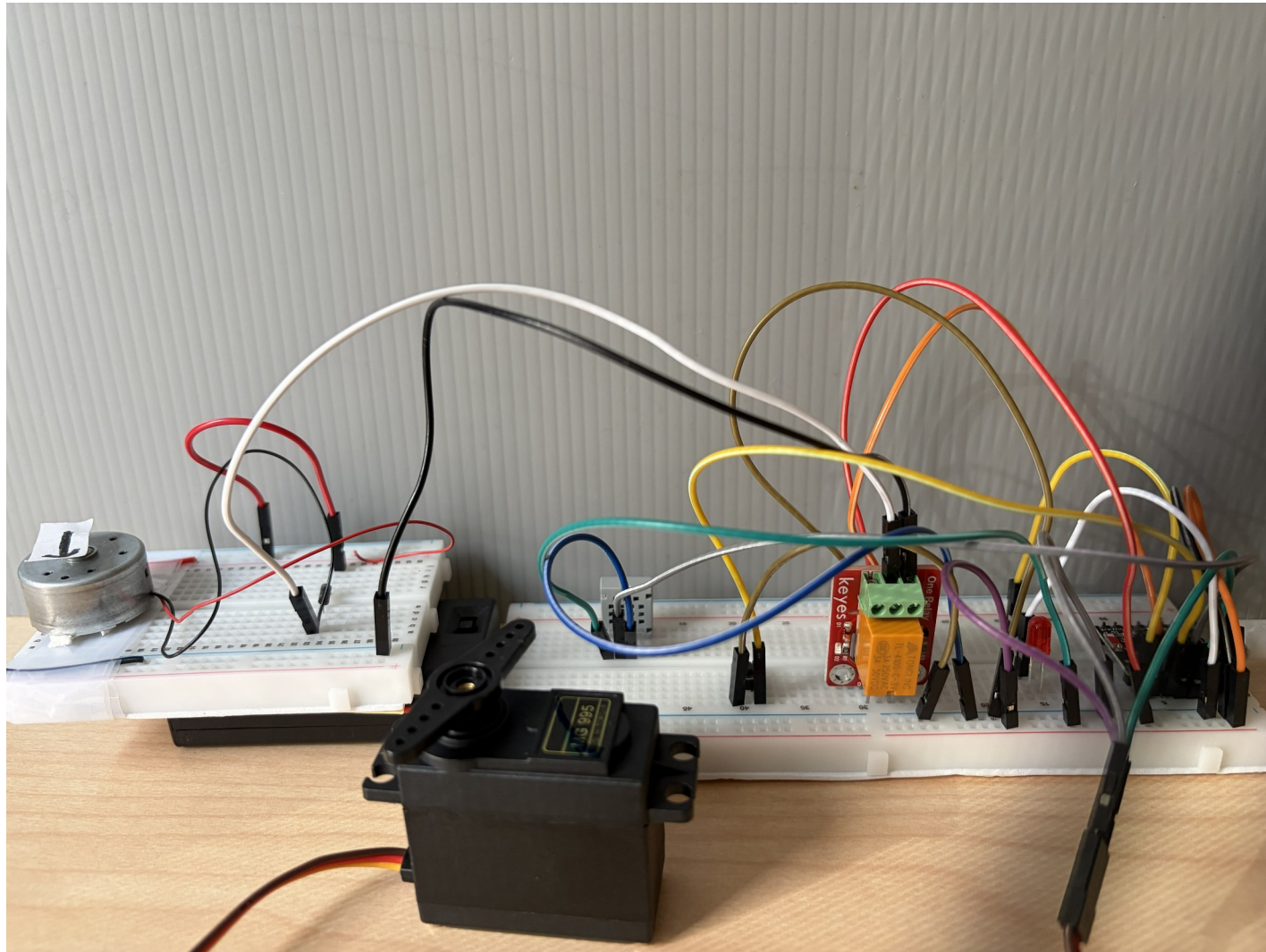
DC MOTOR
RF-300EA



器材清單-5

- 按壓開關





程式下載

- https://github.com/tclintaichung/smart_farmers
- 下載壓縮檔案

AI 協作程式設計

- 請按這裡加入文字

人工智慧應用

- LLM：Large Language Model；大型語言模式，如 ChatGPT、Gemini
- 只要輸入文字，它會搜尋網路、爬梳資料(相當大筆資料)推論出你可能需要的答覆，這資訊不保證一定正確，你得自行判斷
- Copilot：
 - 核心為LLM
 - AI協作
 - Microsoft產品

人工智慧應用：安裝Copilot

- 至Chrome Web Store(<https://chromewebstore.google.com/>)：搜尋Copilot擴充功能(extensions)
- Copilot sidebar for Chrome加入至Chrome瀏覽器



人工智慧應用：

擴充功能

具有完整存取權

這些擴充功能可以查看並變更這個網站上的資訊。

 APK下載器  

 Bing GPT  

 Copilot sidebar for Chrome  



例題1.1

- 試寫一篇作文：參觀木柵動物園
- AI提示：我是國中二年級學生，我參觀木柵動物園，是在周末時間，300字以下文章

例題1.2

- 製作木柵動物園網頁
- AI提示：製作網頁，背景是木柵動物園
- CSS修改
 - `background-image:
url('https://www.gpsmycity.com/img/gd_sight/2870.jpg');`

小結

- 不一定得熟悉HTML或CSS或JavaScript
- 借助Copilot協作，你可以撰寫簡單的網頁

Thonny 軟體

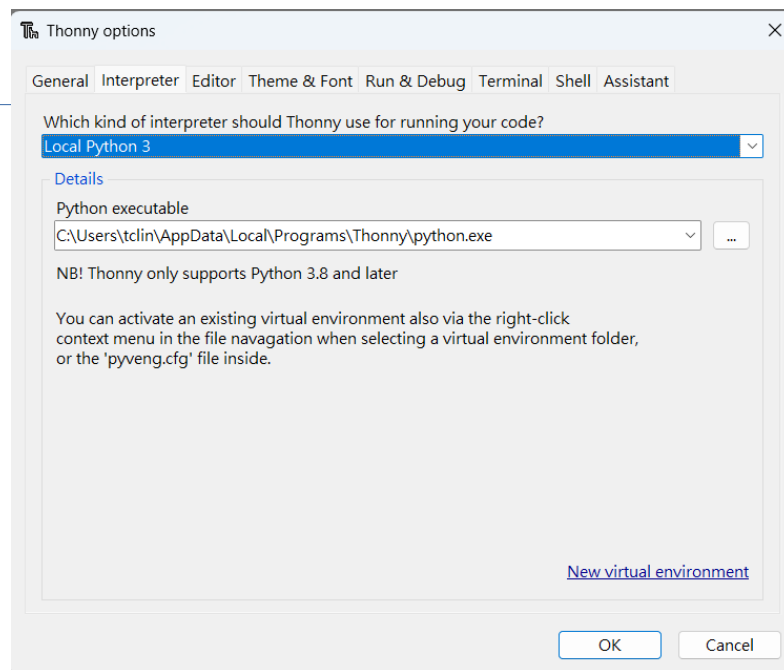
- 請按這裡加入文字

Thonny-1

- Thonny為Python程式開發工具
 - 提供Python與MicroPython直譯器 (interpreter)
 - 可應用至ESP32微控制器控制程式的撰寫
- 安裝Thonny，建立Python開發環境，
 - 官網：<https://thonny.org/>
- Thonny內含Python 3.10，因此並不須另外安裝Python

Thonny-2

- 執行Thonny
 - 設定環境 : Run >> Configure interpreter...
 - ▶ 選Local Python 3



例題2.1

- 寫一個10位同學學籍紀錄，內容有學號、姓名、性別、住址等欄位
- AI提示：寫一個Python程式，10位同學學籍紀錄，內容有學號、姓名、性別、住址等欄位

例題2.2

- 寫Python程式，判斷輸入整數是否為質數
- AI提示：寫Python程式，判斷是否質數

Python 程式設計

- 請按這裡加入文字

Python程式語言介紹

- 前面幾個例題，全部由AI產生，即使不懂Python語法，也可以達成使用者要求的功能
- 介紹一些Python語法，讓各位可以稍稍看得懂程式，也可以進一步修改程式

例題3.1

- Python入門
- AI提示：*Python入門*

Python語法-1

- 變數宣告：不需指定資料型態。變數指定何種資料型態，它就是該資料型態
 - int：整數，例如：`num = 1`
 - float：浮點數，例如：`floatingNum = 1.5`
 - str：字串，例如：`a_string = 'science'`
 - bool：布林值，例如：`isNum = True`
- 可以利用**type**方法確認資料型態

Python語法-2

- 數學運算
 - ▶ `+ - * /`：四則運算
 - ▶ `**`：冪次，例如：`2**3=8`，或`pow(2, 3)`
 - ▶ `%`：模數，即除法後餘數，例如：`9 % 2`餘數為1

Python語法-3

- 判斷式：以「:」分出程式區塊

```
if (....):
```

```
...
```

```
elif (....):  # 相當於else if
```

```
...
```

```
else:
```

```
...
```

Python語法-4

- 輸出與輸入
 - ▶ 輸出：print，如print('Hello')
 - ▶ 輸入：input，如num = int(input('Enter a number:'))

Python語法-5

- 比較、邏輯運算
 - ▶ `>` : 大於
 - ▶ `<` : 小於
 - ▶ `>=` : 大於或等於
 - ▶ `<=` : 小於或等於
 - ▶ `or` : 或閘
 - ▶ `and` : 及閘
 - ▶ `not` : 反相閘

Python語法-6

- 字串處理
 - ▶ `len()`：字串長度，如`a_str = 'short string'`，字串長度為`len(a_str)`
 - ▶ `upper()`：轉為大寫字母，字串組成方法，如`a_str.upper()`
 - ▶ `lower()`：轉為小寫字母，字串組成方法，如`a_str.lower()`
 - ▶ `find()`：尋找匹配字串，字串組成方法，如`a_str.find('o')`，回傳第1個遇到匹配的索引
 - ▶ 字串可以視為字元陣列，如`a_str = 'short string'`，`a_str[0]`為s，0為第1個索引

Python語法-7

- 清單(list)或列表
 - 以[]呈現，相當於陣列，例如：`int_nums = [1, 2, 3]`，或`int_str = [1, 'python']`，可以各式資料型態混合
 - `append`：新增元素
- 元組(tuple)
 - 以()呈現，相當於常數陣列，例如：`const_nums = (1, 2, 3)`，元素不可更動

Python語法-8

- 集合(set)
 - ▶ 以{ }呈現，相當於無重複元素的清單或陣列，無法索引元素，例如：
`num_set = {'C', 'C++', 'python'}`
 - ▶ `add`：新增元素

Python語法-9

- 字典(dictionary)
 - ▶ 也是以{ }呈現，只是每一元素為關鍵詞與值成對(key-value pair)，例如：`student_record = { 'name': 'wang', 'ag': 20 }`
 - ▶ `items()`：出字典所有資料
 - ▶ `values()`：取出字典所有值
 - ▶ `keys()`：取字典所有關鍵詞
 - ▶ `get(關鍵詞)`：取值，例如：`student_record.get('name')`，此亦可以`student_record['name']`取值
 - ▶ 更動值，例如：`student_record['name'] = 'chen'`

Python語法-10

- 重複執行
 - ▶ for-loop :
 - ▶ range函式應用：range(10)產生0至9的整數陣列(不含10)
 - ▶ 例如：for i in range(10):
 print(i)
 - ▶ While-loop
 - ▶ 例如：while i > 10:
 print(i)
 i = i + 1

Python語法-11

- 模組使用
 - 先匯入(import)，例如：產生隨機數模組import random
 - random.random()：產生介於0至1的隨機數
 - random.randint(m, n)：產生介於m至n的隨機整數
- 函式：可供呼叫、重複使用的程式區塊
- #：註解，不執行

Python語法-12

- math模組：數學函式
 - floor(x)：小於或等於x的最大整數
 - ceil(x)：大於或等於x的最小整數
 - fabs：絕對值
 - sin、cos：三角函數值
 - exp：自然指數函數值
 - pow(x, y)：x的y冪次

Python語法-13

- datetime模組
 - `from datetime import datetime`
 - `now = datetime.now()`
- time模組
 - `import time`
 - `time.sleep`：暫停時間，如`time.sleep(2)`，暫停2s
 - `time.localtime`：區域時間

例題3.2

- 輸入三角形高、底，計算面積
- AI提示：輸入三角形高、底，計算面積，函式

例題3.3

- 清單：[1, 2, 3, 4, 5, 6, 7, 8]
- 每一元素平方
- 取得偶數元素
- 計算陣列各元素總和

例題3.4

- 每隔1s顯示目前時間
- AI提示：每隔1s顯示目前時間

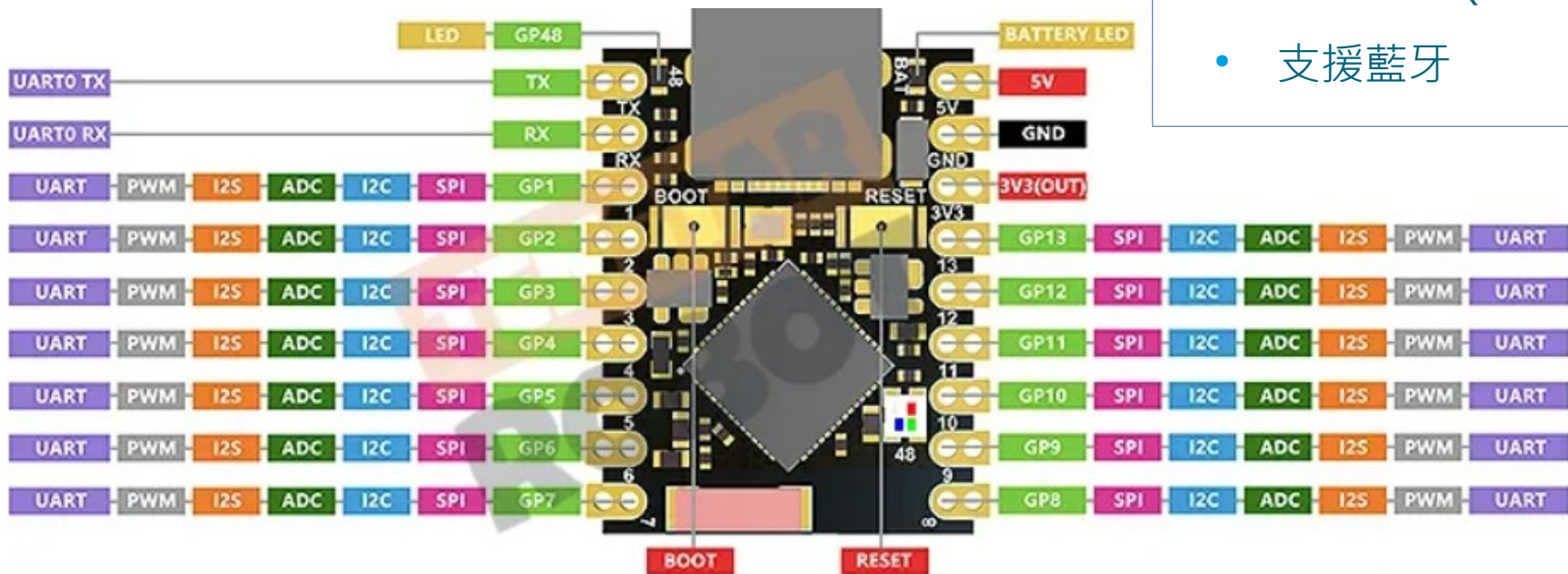
ESP32-S3

- 請按這裡加入文字

ESP32-S3微控制器

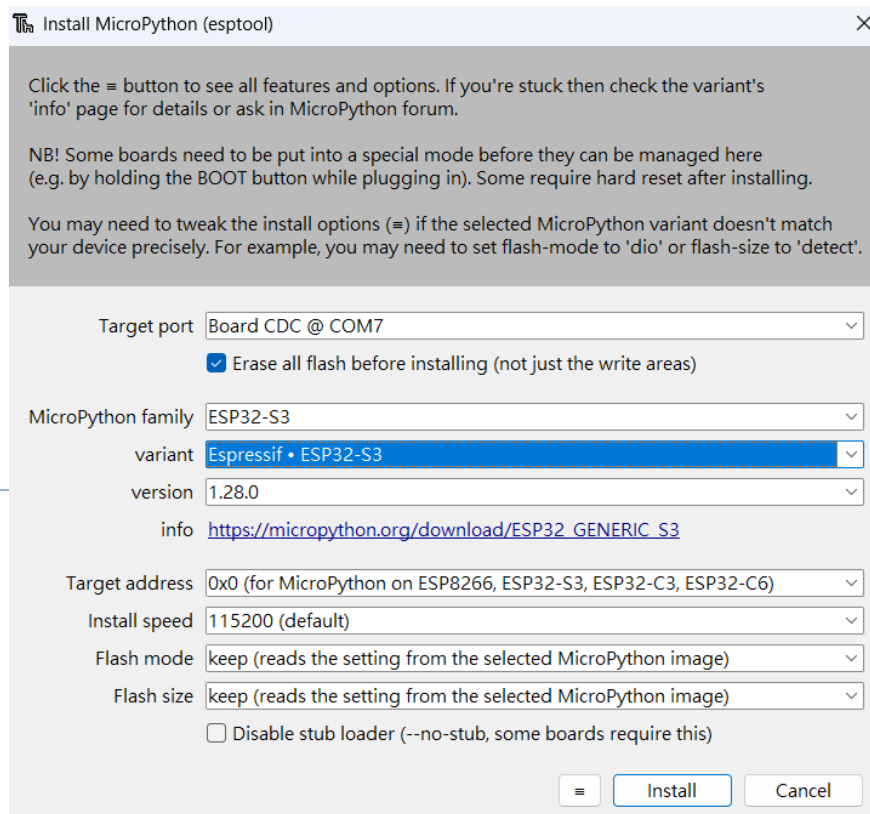
- AI提示：*ESP32-S3 SuperMini*
- AI提示：*ESP32-S3 SuperMini腳位圖*

- 雙核，240MHz
- 4MB FLASH
- 支援WiFi (2.4GHz)
- 支援藍牙



韌體燒錄

- 燒錄韌體至ESP32-S3：點擊Install or update MicroPython (esptool)



Install MicroPython (esptool)

Click the = button to see all features and options. If you're stuck then check the variant's 'info' page for details or ask in MicroPython forum.

NB! Some boards need to be put into a special mode before they can be managed here (e.g. by holding the BOOT button while plugging in). Some require hard reset after installing.

You may need to tweak the install options (=) if the selected MicroPython variant doesn't match your device precisely. For example, you may need to set flash-mode to 'dio' or flash-size to 'detect'.

Target port: Board CDC @ COM7

☒ Erase all flash before installing (not just the write areas)

MicroPython family: ESP32-S3

variant: Espressif • ESP32-S3

version: 1.28.0

info: https://micropython.org/download/ESP32_GENERIC_S3

Target address: 0x0 (for MicroPython on ESP8266, ESP32-S3, ESP32-C3, ESP32-C6)

Install speed: 115200 (default)

Flash mode: keep (reads the setting from the selected MicroPython image)

Flash size: keep (reads the setting from the selected MicroPython image)

☐ Disable stub loader (--no-stub, some boards require this)

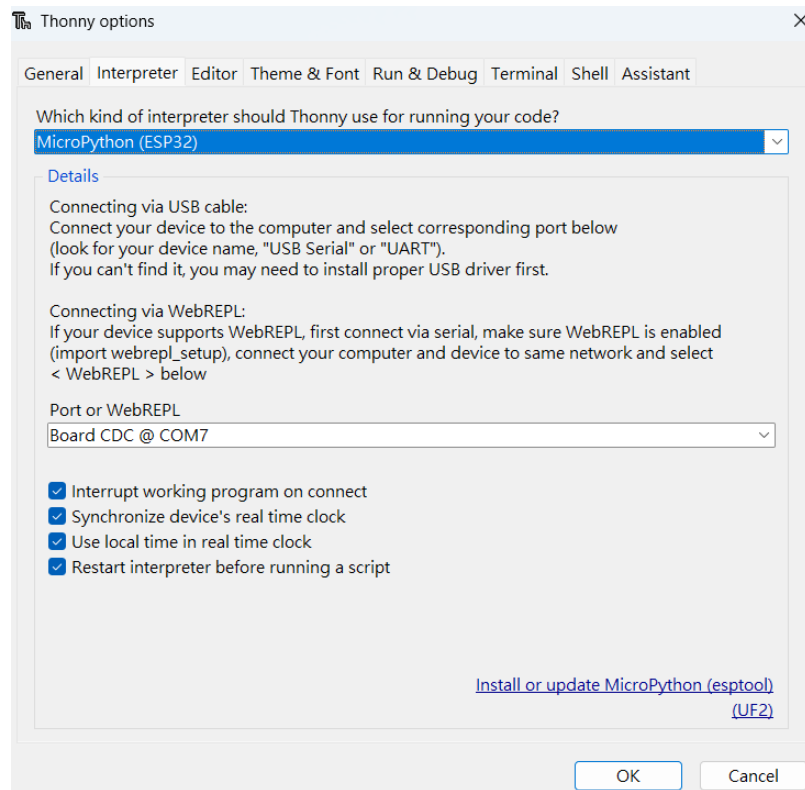
= Install Cancel

第一次燒錄-1

- 按住BOOT
- 按一下RESET
- 放開BOOT
 - 顯示Board CDC @ COM*
- 開始下載程式
- 成功後，下次毋須再按BOOT

第一次燒錄-2

- 執行Thonny
 - ▶ 改變Thonny的Interrupter設定
 - ▶ Run > Configure interrupter , 選MicroPython (ESP32)

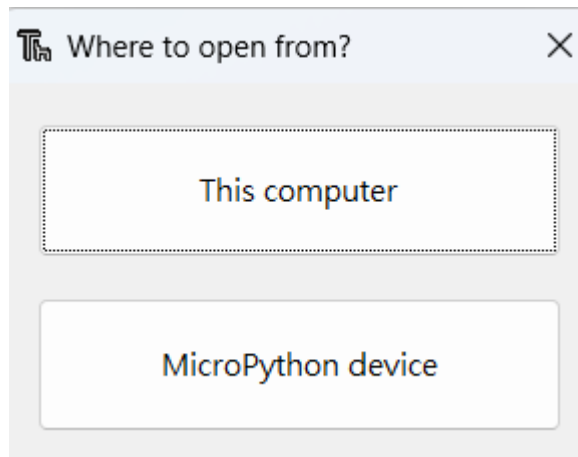


第一次燒錄-3

- 出現Board CDC @ COM7 (按下BOOT後才會出現)
- ESP32-S3晶片內建USB OTG(On-The-Go)，直接用來燒錄程式或串列輸出，不需要額外的USB-to-UART晶片
- 這點功能比傳統ESP32強大

程式編輯、儲存

- 執行Thonny，選擇讀寫檔案所在的裝置，其中MicroPython device即為ESP32-S3，注意此ESP32-S3已經完成韌體燒錄
- 可以在PC端編輯、執行程式，再上傳至ESP32-S3
- 停止執行，按



例題4.1

- 利用ESP32的Pin8接LED、330 Ω 電阻，間隔1s亮暗一次
- AI提示：*esp32 pin8 led python*

程式說明

- machine模組、Pin函式：設定GPIO，2個引數；腳位、輸出(Pin.OUT)或輸入(Pin.IN)腳位，例如：`led = Pin(8, Pin.OUT)`，第8腳位為輸出腳位
 - `led.value(1)`：輸出高準位(3.3V)
 - `led.value(0)`：輸出低準位(0V)
- time模組：提供sleep函式
 - `from time import sleep`
 - `sleep(0.5)`：暫停500ms

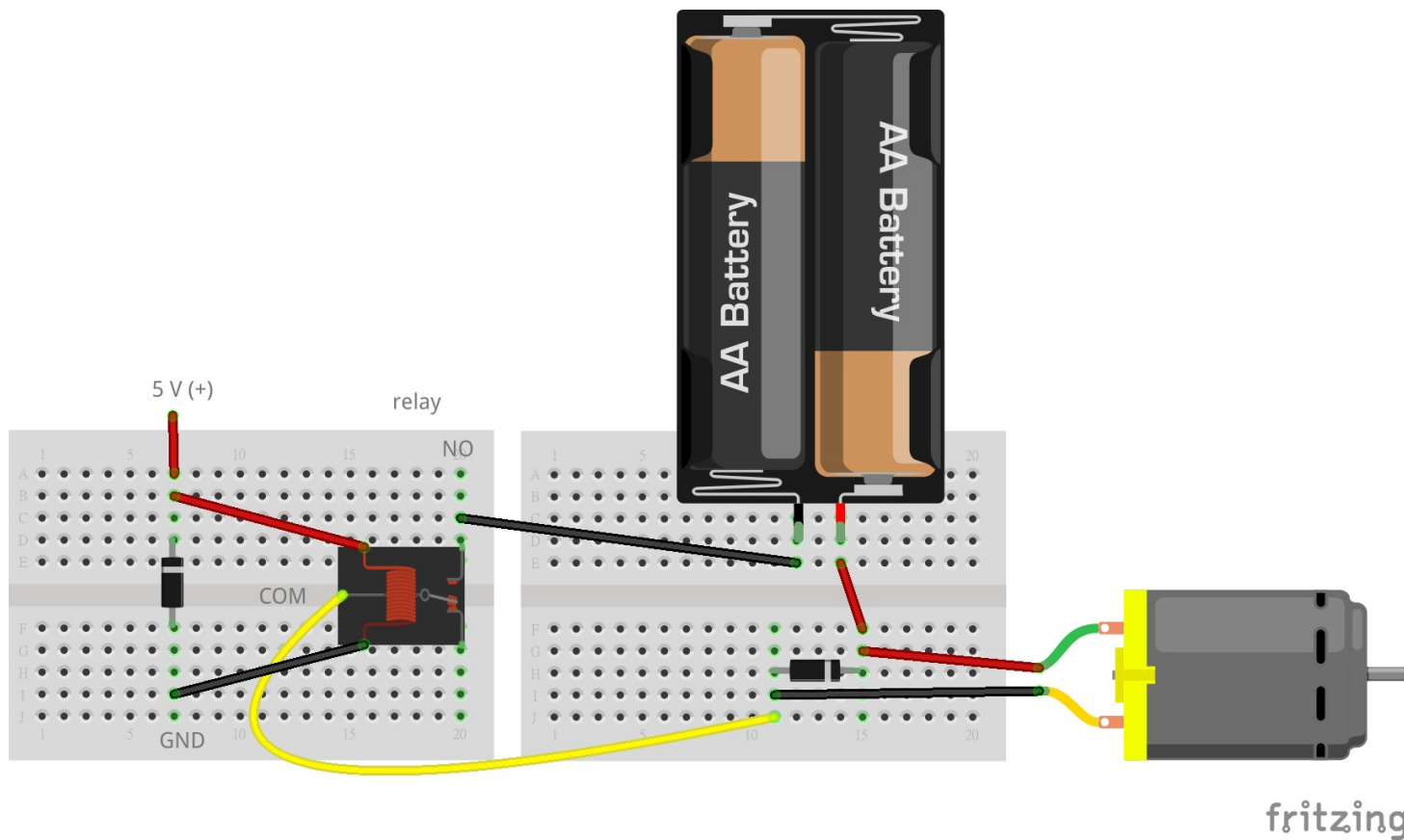
例題4.2

- 利用按壓開關控制繼電器模組，進一步控制DC馬達轉動：按下開關馬達轉動、鬆開停止
- AI提示：**esp32 relay button press on release off python**

電路布置-1

- 馬達紅線接VCC正極(3V)，VCC負極接繼電器接點(NO，常開接點)，繼電器COM接馬達負極
- 1N4007二極體陰極接馬達正極，陽極接馬達負極。此作用在於消耗馬達產生的反電動勢，保護ESP32-S3；繼電器模組已設二極體
- 繼電器信號輸入接Pin9
- 按壓開關一端接GND，一端接Pin10

電路布置-2



- `relay_pin = Pin(9, Pin.OUT)`，第9腳位為輸出腳位
- `button_pin = Pin(10, Pin.IN, Pin.PULL_UP)`，第10腳位為輸入腳位，使用內部提升電阻

例題4.3

- 利用ESP32量測室內溫度、濕度，直接顯示測量值
- AI提示：*esp32 with DHT22 to measure temperature and humidity*

電路布置

- AM2120接5V、Pin11、GND

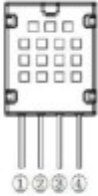
Pin	Color	Name	Description	
1	Red	VDD	Power (3.3V-5.5V)	
2	Yellow	SDA	Serial data, two-way port	
3	Black	GND	Ground	
4		NC	No Connection	

Table 4 Interface definition description

- AM2120為溫濕度感測模組，類似DHT22、DHT11，AM2120量測更準確，可以直接使用dht模組

程式說明

- `sensor = dht.DHT22(Pin(11))`：配置Pin11接DHT22信號腳位
- `sensor.measure()`：量測溫濕度
- `sensor.temperature()`：溫度值
- `sensor.humidity()`：濕度值

例題4.4

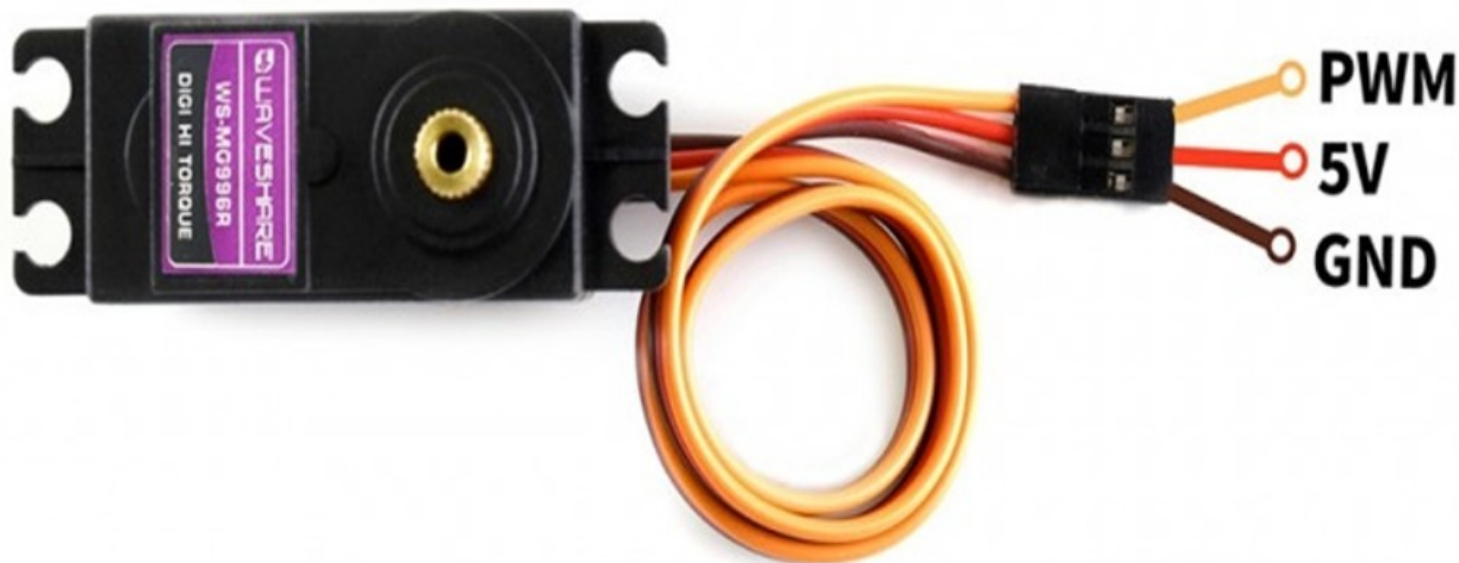
- 利用ESP32控制MG995伺服馬達
- PWM：Pulse Width Modulation，脈衝寬度調變
- AI提示：**PWM**
- AI提示：**diagram**

Servo Motor

- 50 Hz，週期20ms
- 脈衝寬度0.5ms (高電位持續0.5ms)，相當於0°
- 脈衝寬度2.5ms，相當於180°
- duty cycle：佔空比，脈衝寬度佔整個週期的比率
- AI提示：**esp32 pin12 control mg995**

電路布置

- MG995三條線：紅線接5V、棕線GND、橘線接Pin12



程式說明

- `servo = PWM(Pin(12))`：配置Pin12接PWM信號腳位
- `servo.duty`：設定佔空比，0~1023
- `ms` = 脈衝寬度
- `duty = int(ms / 1000 * 1023 * 50)`
- 佔空比100%，相當於1023

例題4.5

- 利用ESP32控制LED亮暗
- AI提示：*esp32 pwm led lightness control pin8*

程式說明

- `pwm.freq(1000)` : PWM信號頻率1000Hz
- `pwm.duty` : `duty`等於0最暗，`duty`等於1023最亮

MQTT

- 請按這裡加入文字

MQTT-1

- MQTT(Message Queuing Telemetry Transport)
 - 建立物聯網的重要工具
 - 機器與機器(M2M)或物與物之間通訊協定
 - 相較於http協定，它顯得簡單、輕量
- MQTT由三個成員構成：
 - 「發布者」 (publisher)
 - 「訂閱者」 (subscriber)
 - 「伺服器」或「代理人」 (broker)

MQTT-2

- 三者可以在同一部裝置，也可以分散至各個裝置
 - ▶ 裝置如：樹莓派、筆電、ESP32-S3模組、或雲端伺服器
 - ▶ 「發布者」將訊息發布至「伺服器」
 - ▶ 「訂閱者」自「伺服器」取得相關資訊



MQTT-3

- 以YouTube比擬，YouTube網站為「伺服器」，網民將影片上傳到「伺服器」供人觀看，他就是「發布者」；如果觀看者喜歡「發布者」的相關影片，成為「訂閱者」，只要新影片上傳，都會收到通知
- MQTT與YouTube的差異：
 - MQTT是以訂閱主題發送訊息
 - YouTube是針對特定「發布者」，無論任何主題都會通知「訂閱者」

MQTT-4

- 所有傳遞訊息的形式，常使用的項目
 - topic(主題)
 - payload(負載)
- topic與payload都是字串，topic由上而下階層式歸類
 - 舉例說明：發布topic='cmd'、payload='1'訊息，只要連上MQTT伺服器，訂閱'cmd'，就可以獲得'1'訊息負載，藉此傳遞控制裝置指令

MQTT-5

- topic與payload...
 - 如果有好幾個裝置控制指令，例如：電燈開關Switch 1、Switch 2，主題topic='home/room1/sw1'、topic='home/room1/sw2'、topic='home/room2/sw1'、topic='home/room2/sw2'，上層與下層主題以'/'分隔，允許更多層主題
 - +：單層萬用字元，如topic='home/+/sw1'
 - #：多層萬用字元，如topic='home/#'

MQTT-6

- 「mosquitto」MQTT伺服器，它是根據MQTT通訊協定3.1與3.1.1版 實作的輕量型開源訊息伺服器（ open source message broker ），從低功率單板機到伺服器的裝置均適用
 - <https://mosquitto.org/>
- 安裝mosquitto：安裝在個人電腦或筆電，作業系統為Windows
 - 註：mosquitto 比蚊子（ mosquito ）多1個t

MQTT-7

- 下載安裝檔案並執行安裝： 64 位元版本

<https://mosquitto.org/files/binary/win64/mosquitto-2.0.21a-install-windows-x64.exe>

- 安裝目錄C:\Program Files\mosquitto

MQTT-8

- 新增路徑：如果要在任何目錄下執行MQTT指令，就必須新增環境變數
 - ▶ 進入「Windows設定」>「系統」>「關於」(系統資訊)>「進階系統設定」>「環境變數」，編輯「使用者變數」Path，新增C:\Program Files\mosquitto

MQTT-9

- 測試：利用命令提示字元測試MQTT（視窗1），發布與訂閱cmd主題
- 啟動 MQTT伺服器
 - > mosquitto
- 訂閱訊息：先設定訂閱主題
 - > mosquitto_sub -t cmd
 - t：設定主題

MQTT-10

- 發布訊息：打開另一個「命令提示字元」視窗，發布主題、訊息
 > mosquitto_pub -t cmd -m 1
 -m：設定主題內容

MQTT-11

- 編輯mosquitto.conf：目錄C:\Program Files\ mosquitto
- 新增以下3行

```
protocol mqtt  
listener 1883  
allow_anonymous true
```

- 不同IP可以進行MQTT通訊
> mosquitto_sub -h 192.168.0.156 -t "temp"

雲端MQTT伺服器

- broker.hivemq.com，此為免費伺服器
- 訂閱主題
 - > mosquitto_sub -h broker.hivemq.com -t "temp"
 - h：設定伺服器，任何人發布主題為temp，都會接收到
- 發布主題
 - > mosquitto_pub -h broker.hivemq.com -t "temp" -m "hello"
- 若MQTT伺服器設在本機，-h localhost

umqtt模組-1

- 用於ESP32-S3
- 應用umqtt模組進行ESP32-S3的MQTT訊息傳遞
- simple2.py與robust2.py模組，分別至以下網址下載：

<https://github.com/fizista/micropython-umqtt.simple2/tree/master/src/umqtt> 下載simple2.py

<https://github.com/fizista/micropython-umqtt.robust2/tree/master/src/umqtt> 下載robust2.py

umqtt模組-2

- 執行Thonny，View>>files，可以瀏覽本機與MicroPython device (即ESP32-S3)
- 在ESP32-S3新增umqtt目錄，複製simple2.py、robust2.py
 - 更直接方式，先下載至PC目錄umqtt，再上傳至ESP32-S3

umqtt模組-3

- 匯入MQTTClient模組—`from umqtt.robust2 import MQTTClient`
- umqtt類別與函式：
 - MQTTClient類別：用於建立MQTTClient物件，引數為客戶帳號與MQTT伺服器網址，例如：`client = MQTTClient(mqtt_client, mqtt_server)`
 - connect：連接MQTT伺服器，回傳True表示連線中，例如：`client.connect()`

umqtt模組-4

- ▶ `subscribe`：訂閱MQTT主題，引數為主題，例如：
`client.subscribe(topic)`
- ▶ `publish`：發布MQTT訊息，4個引數分別為主題、訊息、`retain`、`qos`，後2個為關鍵詞引數，可以忽略；其中`retain`預設`False`，保存所有未傳遞訊息，若設為`True`，只保留最近一筆訊息，其餘刪除，`qos`預設0，例如：`client.publish(topic, message)`

umqtt模組-5

- ▶ `is_conn_issue`：如果成功連上MQTT伺服器，不會回傳錯誤訊息；若回傳錯誤訊息，呼叫`reconnect()`
- ▶ `set_callback`：設定回呼函式，接收到MQTT伺服器轉傳來的訊息時，執行回呼函式(callback function)，回呼函式需有4個引數：主題、訊息、retain、dup，後2個雖然沒用到，仍須維持4個引數格式，例如：`client.set_callback(callback_function)`

主題格式

- smart_farmer學號/temp_humi : 溫度/濕度
 - 如smart_farmer00/temp_humi
- smart_farmer學號/fan
 - 如smart_farmer00/fan
 - 利用前面馬達控制 (Pin 9)
 - 亦可利用LED (Pin 8)

例題5.1

- ESP32-S3設LED，在PC端以MQTT方式控制LED的亮暗
- AI提示：ESP32設LED，在PC端以MQTT方式控制LED的亮暗

程式說明-1

- 下載的umqtt，將目錄上傳至ESP32-S3
- ESP32-S3端：LED接ESP32-S3 Pin8
- PC端
 - > mosquitto_pub -t smart_farmer00/fan -m ON
 - > mosquitto_pub -t smart_farmer00/fan -m OFF
- ESP32-S3訂閱所有主題#，會接收到伺服器發布的所有主題

程式說明-2

- 延伸題：使用雲端MQTT伺服器broker.hivemq.com。(ex5-1_ext.py)

```
client = MQTTClient("esp32_client", "broker.hivemq.com",  
user="", password="")
```

```
client.subscribe("smart_farmer00/fan")：需設訂閱主題
```

- PC端

```
> mosquitto_pub -h broker.hivemq.com -t smart_farmer00/fan -m  
ON
```

例題5.2

- ESP32-S3 量測溫度，以MQTT方式上傳至PC端MQTT伺服器
- AI提示：ESP32 量測溫度，以MQTT方式上傳至PC端MQTT伺服器

程式說明

- `payload = f'temp={temp}/humi={humi} '`
- `client.publish('smart_farmer00/temp_humi', payload)`：**主題為**`temp_humi`
- PC**端**：**假設**`IP = 192.168.0.121`，**與**`ESP32-S3`**不同**`IP`

`> mosquitto_sub -h 192.168.0.121 -t smart_farmer00/temp_humi`
- **延伸題**：**使用雲端MQTT伺服器**`broker.hivemq.com`

藍牙低功耗

- 請按這裡加入文字

藍牙低功耗應用

- 「藍牙低功耗」(BLE) 為「藍牙技術聯盟」(Bluetooth Special Interest Group ; Bluetooth SIG) 發展的個人區域網路技術，應用於行動裝置，例如：智慧型手機、智慧手環、或耳機等之間的通訊，低功耗、低成本是主要特色
- 使用BLE通訊，**毋須支付網路使用費**，即可具備無線通訊的功能，傳輸距離甚至達100m
- 對於只需要在小區域建立物聯網，資料傳輸量少的應用場合，BLE具有相當大的優勢
- ESP32-S3支援BLE

藍牙低功耗傳輸距離

- 傳輸距離
- AI提示：*esp32-S3 ble distance*

MicroPython藍牙低功耗

- AI提示：*ESP32 ble micropython led*
- AI提示：*ESP32 micropython BLE*

ESP32-S3與智慧型手機BLE通訊

- ESP32-S3主要用來感測訊號或控制其他設備，而智慧型手機發出讀取ESP32-S3發出的訊號或對它下指令請求
 - ESP32-S3 是伺服器 (server)，而手機視為用戶端 (client)
- 智慧型手機可以連上多個ESP32-S3，例如：一個ESP32-S3負責量測溫濕度，另一個負責控制電燈等，ESP32-S3只是環繞在智慧型手機周圍眾多裝置之一
 - 智慧型手機為中心裝置 (central)，ESP32-S3為周邊裝置 (peripheral)
- 每個BLE裝置都有一個唯一的媒體存取控制位址 (Media Access Control Address ; MAC Address) 用來確認它的身分
- BLE 通訊開始時，ESP32-S3會持續進行「服務廣告」，讓鄰近BLE裝置 (中心裝置) 知道它的存在並連結

BLE通訊

- **GATT** 是Generic Attribute Profile 的首字母縮略詞，翻譯為「通用屬性配置文件」，它是BLE核心資料交換框架，定義設備之間如何組織、讀寫、通知資料，是BLE的資料庫語言
- 配置文件(Profile)：一個或多個服務構成配置文件
- 服務(Service)：功能集合，如電池服務，一個或多個特徵組合成服務，每一個「服務」有一個通用唯一識別碼 (Universally Unique Identifier ; UUID)
- 特徵(Characteristic)：可視為資料點(data point)，如電池電量，每一個「特徵」有UUID，特徵屬性有
 - 性質(property)：如該資訊屬於WRITE、READ、或NOTIFY等
 - 值(value)：資訊內容，如量測溫度值
 - 描述符(descriptor)：記載特徵性質，如溫度單位Celsius或Fahrenheit

UUID-1

- 在BLE應用裡，「服務」與「特徵」UUID的設定是重點
 - UUID分長碼與短碼，長碼為128-bit，由32個0~F（十六進位數）組成字串，短碼為16-bit整數
- Bluetooth SIG 預留一定範圍的UUID用於標準屬性，如0x1809用於健康體溫、0x180D用於心率監測、0x180F用於電池電量

UUID-2

- 智慧型手機與 ESP32-S3 進行BLE通訊，利用串列傳輸（UART）介面，使用 Nordic UART Service UUID (NUS UUID)
 - UART服務UUID：'6E400001-B5A3-F393-E0A9-E50E24DCCA9E'
 - RX特徵UUID：'6E400002-B5A3-F393-E0A9-E50E24DCCA9E'
 - TX特徵UUID：'6E400003-B5A3-F393-E0A9-E50E24DCCA9E'
- RX或TX特徵的性質
 - 站在ESP32-S3的視角觀之，RX是ESP32-S3接收資訊，所以手機是寫入資料(WRITE)，同理，TX是ESP32-S3傳出資訊，所以手機是讀取資料(READ)

NUS

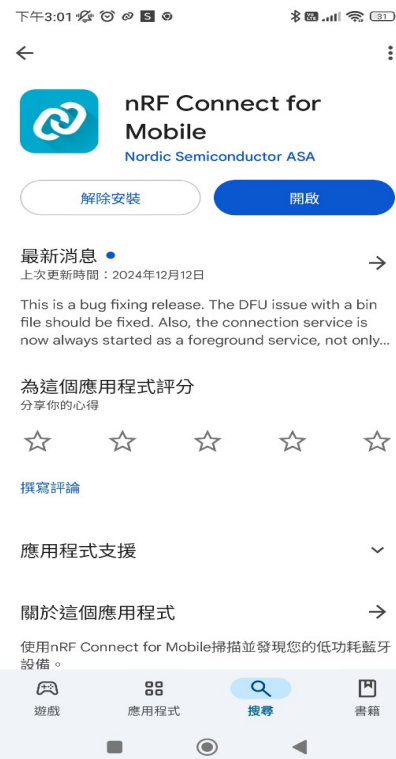
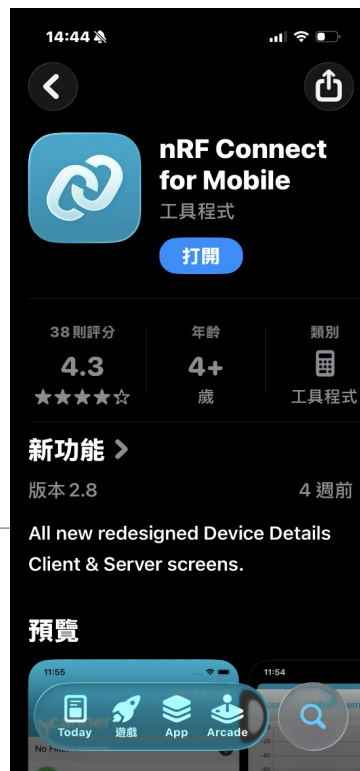
- Nordic UART Service (NUS)
 - Nordic半導體公司自訂GATT，用BLE模擬傳統UART進行串列通訊

UUID-3

- 或上 <https://www.uuidgenerator.net>，隨機產生128-bit的長碼UUID，避免UUID重疊
- 除使用 NUS UUID，也將使用16-bit與128-bit自訂UUID

手機APP – nRF Connect

- 下載安裝Androi或iOS手機APP：nRF Connect
- 開啟藍牙設定，可以進行BLE通訊



ubluetooth-1

- MicroPython提供BLE 模組ubluetooth，可以應用於ESP32-S3的 BLE 通訊
- ubluetooth模組詳細資訊：<https://docs.micropython.org/en/latest/library/ubluetooth.html>
- 匯入模組
 - ubluetooth模組：`import ubluetooth`

ubluetooth-2

- BLE類別：用於建立BLE物件
 - `ble = ubluetooth.BLE()`
 - `ble.active(True)`：啟用BLE裝置
- UUID 類別：用於建立UUID物件，第1引數為128-bit UUID字串或16 bit 整數，第2引數為旗標，有寫入旗標`FLAG_WRITE`、讀取旗標`FLAG_READ`、通知旗標`FLAG_NOTIFY`
- `ble.gatts_register_services`：註冊服務，引數為服務與特徵的UUID

ubluetooth-3

- **ble.irq**：中斷事件處理函式
 - 事件編號為1(_IRQ_GATTS_CONNECT)：手機連上BLE裝置
 - 事件編號為2(_IRQ_GATTS_DISCONNECT)：手機中斷連接BLE裝置
 - 事件編號為3(_IRQ_GATTS_WRITE)：當特徵被寫入時觸發中斷
- **ble.gap_advertise**：廣告服務，讓鄰近BLE 裝置知道所提供的服務，第1個引數為每廣播一次的時間，單位為 μs ，第2個引數為廣播內容

例題6.1

- 利用ESP32-S3的 藍牙低功耗功能廣播服務，再以手機輸入指令控制LED
 - ON : LED亮
 - OFF : LED暗
- 使用NUS UUID
- AI提示：*esp32 ble NUS UUID control LED python ubluetooth*

程式說明：服務廣播-1

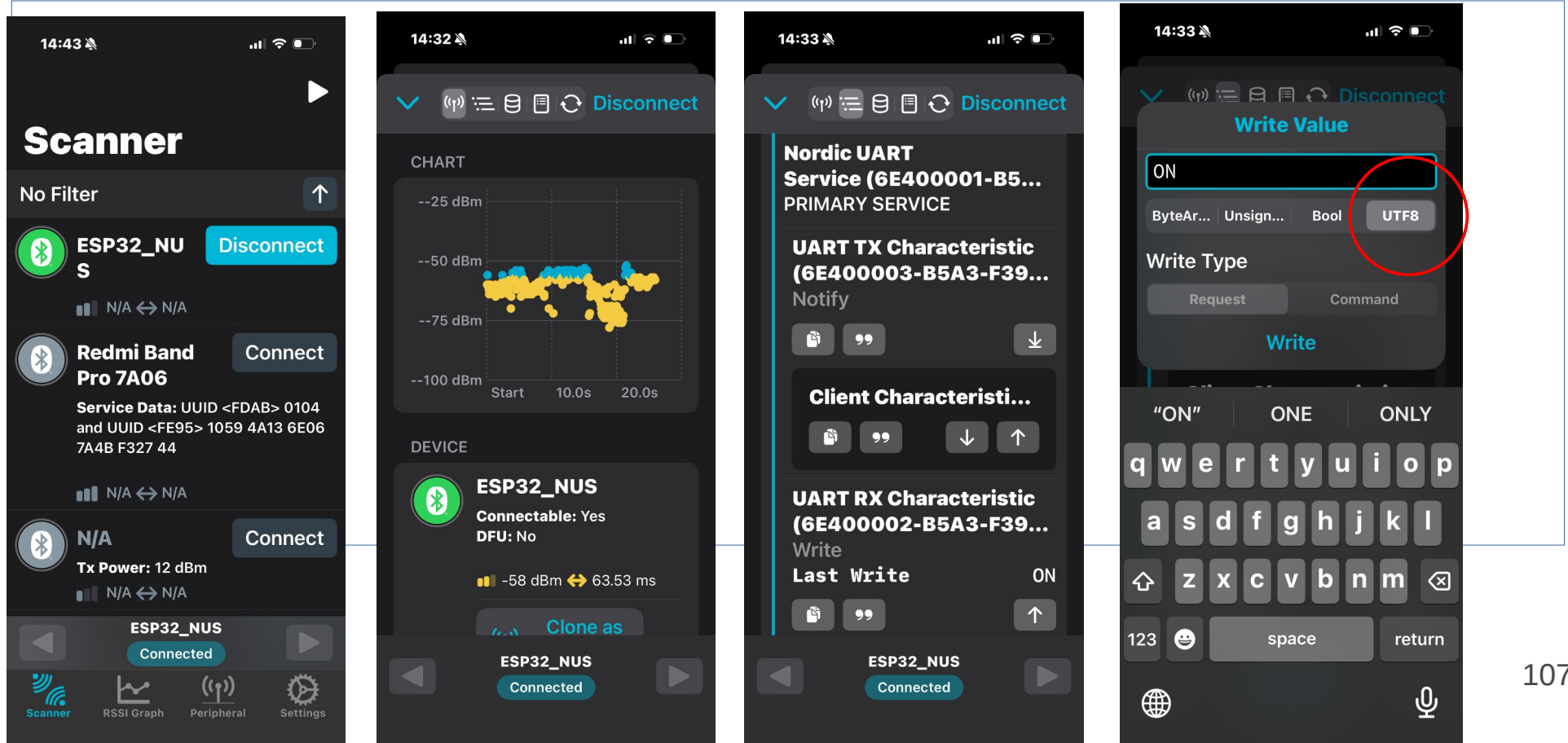
- `adv_data = bytes([0x02, 0x01, 0x06, 0x07, 0x09,]) + b"ESP_00"`
 - `0x02, 0x01, 0x06`：長度2，內容第1個為旗標，第2個為此BLE裝置一般可被發現模式(`0x02`)，但不接受傳統藍牙(`0x04`)，兩相加等於`0x06`
 - `0x0A, 0x09, b"ESP_00"`：BLE裝置名稱"ESP_00"，`0x09`顯示完整名稱(Complete Local Name)，此名稱會出現在手機APP搜尋列中，名稱長度6，兩項加起來共7個位元組(`0x07`)
 - 可隨個人學號設定裝置名稱，如ESP_01

程式說明：服務廣播-2

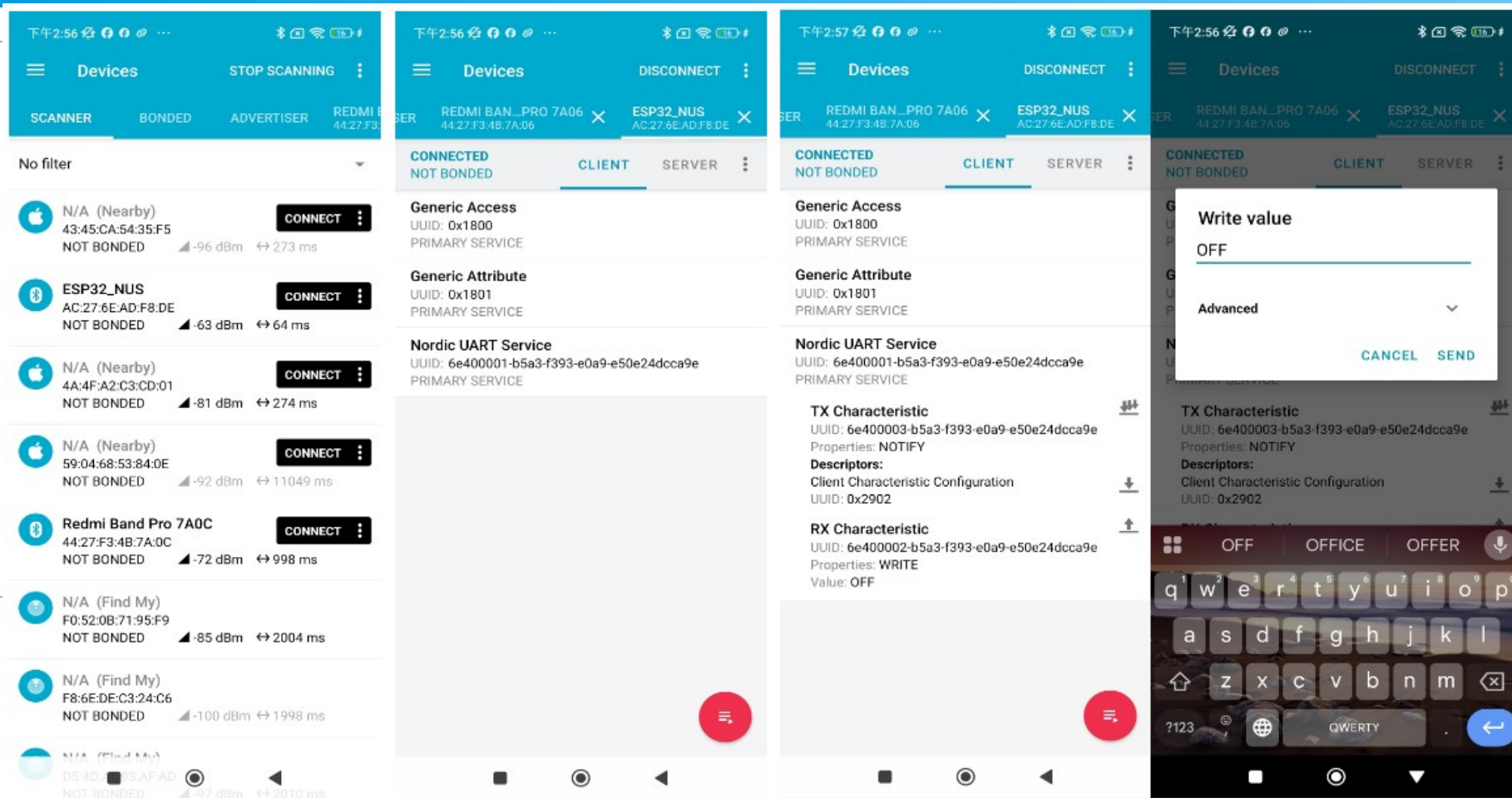
- 旗標(Flags)值

Bitmask	Meaning (English)	中文說明
0x01	LE Limited Discoverable Mode	限時可被發現模式
0x02	LE General Discoverable Mode	一般可被發現模式
0x04	BR/EDR Not Supported (BLE-only)	不支援傳統藍牙 (僅 BLE)
0x08	Simultaneous LE + BR/EDR (Controller)	同時支援 BLE 與傳統藍牙 (控制器)
0x10	Simultaneous LE + BR/EDR (Host)	同時支援 BLE 與傳統藍牙 (主機)

iPhone手機執行nRF Connect



Remi手機執行nRF Connect



例題6.2

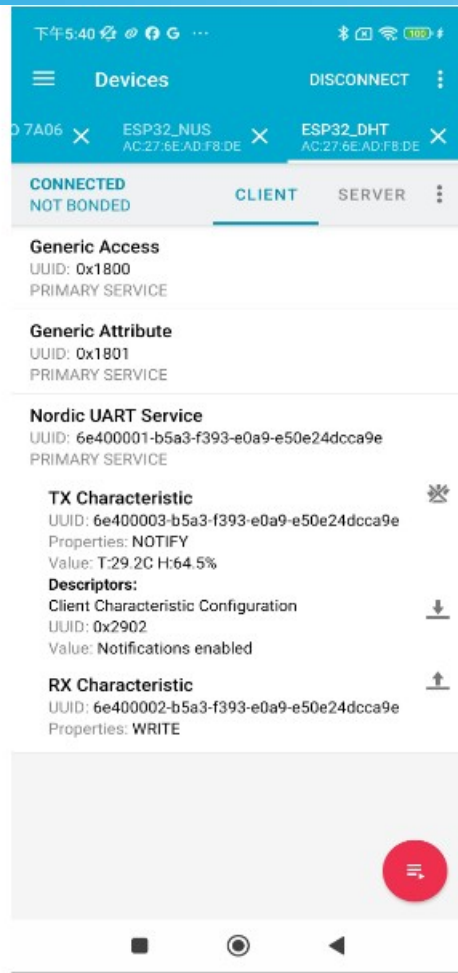
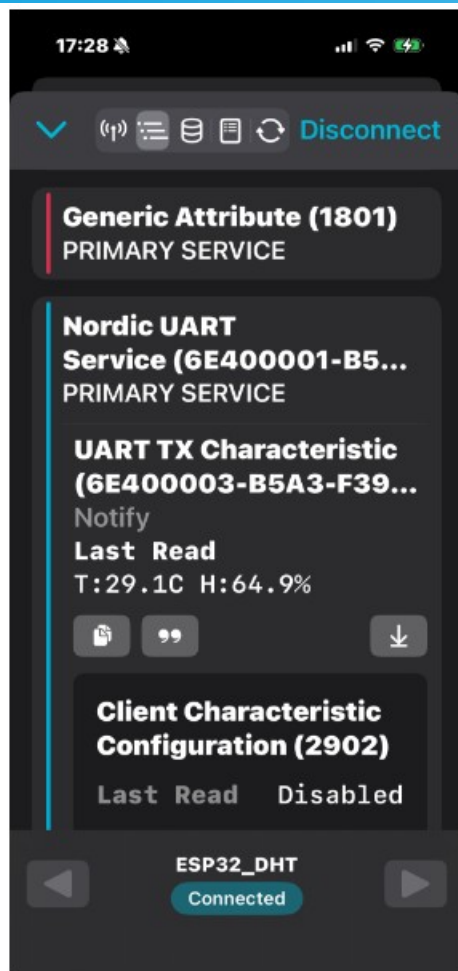
- 重作例題6.1，使用自訂UUID：
 - 服務：0x1800
 - RX：0x2A01
 - TX：0x2A02
- 程式說明：僅需改變UUID
- iOS不支援訊息16 bits UUID

例題6.3

- 利用ESP32-S3將感測溫濕度傳送出去，以手機讀取溫濕度
- 註：結合例題5.2與例題6.1

手機顯示

- 請按這裡



ESP32-S3獨立作業

- 將例題檔案名稱改為main.py，如ex6-3.py改存main.py
- ESP32-S3不再連接電腦，以行動電源USB Type-C電纜線連接供電
- ESP32-S3重置後，會先執行boot.py (可以設連接網路程式，若無特別需求，可暫保留不更動內容)，接著會執行main.py

使用ESP32-S3注意事項

- 數位輸入的電壓不可超過3.3V
- 切記不可輸入電壓至GND，若是誤操作可能會燒毀板子
- 對照腳位圖，確定接線

各位辛苦了，謝謝參與
今日的學習，也期望各
位有所收穫！

若有問題可email我：tclinnchu@gmail.com，我
會樂意盡速回覆你

